FrameSpace — описание проекта и текст защиты

Контрольная работа №3 · ветка `spo/kr3-spring-backend` · репозиторий framespace_kinostudio

1. Краткое описание проекта

FrameSpace — информационная система киностудии для учёта заявок на съёмку. В КР №3 реализована **серверная часть на Spring Boot**: REST API, PostgreSQL, миграции Flyway, JWT-авторизация, Docker Compose и встроенный web-интерфейс для проверки.

Предметная область та же, что в КР №1 и КР №2: **клиенты** (заказчики), **локации** (павильоны, студии, натура, офисы), **заявки на съёмку** со статусами, бюджетом, длительностью и приоритетом. JavaFX-клиент из КР №2 остаётся отдельным; этот backend — общая точка для будущих клиентов, в том числе Android в КР №4.

Запуск: `./backend/start-compose.sh` → UI на `http://localhost:8080/`, Swagger — `/swagger-ui.html`, API — `/api`.

2. Связь контрольных работ

КР	Что было	Что взято в КР №3
КР №1	Модель БД: клиенты, локации, заявки, связи 1:N	Таблица `client`, тип локации, seed-данные
КР №2	JavaFX, граф статусов, правила дат и ролей	Граф на сервере, проверка дат, роли VIEWER / OPERATOR / ADMIN
КР №3	Spring Boot backend	REST, безопасность, миграции, UI для демо

Шаблон Student Hub перенесён в предметную область киностудии (таблица соответствия — в `README_KR3.md`): Student → `ShootingRequest`, StudyGroup → `Location`, StudentStatus → `RequestStatus`, RiskLevel → `PriorityLevel`, TEACHER → `OPERATOR`, плюс сущность **Client**.

3. Стек и архитектура

Стек: Java 21, Spring Boot 3, Spring Web, Spring Security, Spring Data JPA / Hibernate, Flyway, PostgreSQL, JWT (access + refresh), springdoc OpenAPI, Docker Compose, статический UI (`index.html` + `app.js`).

Слои (пакет `ru.mirea.framespace`):

1. **controller** — HTTP, роли, валидация входа
2. **service** — бизнес-правила
3. **repository** — доступ к БД
4. **domain** — сущности и enum
5. **dto** — контракт API
6. **security / config / filter / error** — JWT, настройки, умный фильтр, единый формат ошибок

Точка входа: `backend/src/main/java/ru/mirea/framespace/FrameSpaceApplication.java`. Конфиг: `backend/src/main/resources/application.yml` — Hibernate `ddl-auto: validate` (схему меняет только Flyway), `open-in-view: false`.

4. Текст защиты (устная речь)

Вступление

Здравствуйте. Тема КР № 3 — backend информационной системы киностудии FrameSpace. Цель: перенести модель и бизнес-правила с клиентского приложения на сервер, чтобы данные и ограничения были едиными для любых клиентов API.

Я покажу: структуру проекта, схему БД и миграции, сущности и правила статусов, REST API и роли, безопасность, web-интерфейс, тесты.

Архитектура и запуск

Проект в каталоге `backend/`. Зависимости — в `backend/pom.xml`: Web, Security, Data JPA, Validation, Flyway, PostgreSQL, springdoc.

Инфраструктура — `backend/docker-compose.yml`: приложение и PostgreSQL. Переменные — `.env` по образцу `.env.example` (БД, JWT, bootstrap-админ).

После старта доступны UI, Swagger и health-check Actuator. Первый вход под bootstrap-админом требует смены временного пароля — это задумано в `BootstrapAdmin` и фильтре смены пароля.

База данных и миграции Flyway

Схема версионируется в `backend/src/main/resources/db/migration/`. Hibernate только **проверяет** схему, не создаёт её сам — так нельзя «случайно» разъехаться с продом.

V1 (`V1__initial_schema.sql`) — базовая схема: пользователи, refresh-сессии, локации, заявки, теги, заметки, аудит.

V2 (`V2__clients_and_location_type.sql`) — приведение к модели КР №1:

- таблица `client` (ФИО, уникальный email в нижнем регистре, телефон, version);
- перенос клиентов из полей заявки по email (для старых баз без потери данных);
- в заявке `client_id` NOT NULL, FK `ON DELETE RESTRICT` (клиента с заявками удалить нельзя);
- у локации поле `type` с CHECK: PAVILION, OUTDOOR, STUDIO, OFFICE;
- CHECK на статус заявки и вместимость.

V3 (`V3__kr1_seed_data.sql`) — тестовые данные КР №1: 5 клиентов, 8 локаций (коды PAV-1… CONF-HALL), 10 заявок `KR1-001` … `KR1-010`. `ON CONFLICT DO NOTHING` — повторный запуск не затирает ручные данные.

Доменная модель

Сущности в `backend/src/main/java/ru/mirea/framespace/domain/`:

Файл	Зачем
`Client.java`	Заказчик съёмки; связь 1:N с заявками
`Location.java` + `LocationType.java`	Площадка и её тип
`ShootingRequest.java`	Заявка: клиент, локация, дата, бюджет, статус, приоритет, теги, архив
`RequestStatus.java`	Enum и граф переходов
`PriorityLevel.java`	Уровень приоритета
`RequestNote.java`	Заметки к заявке
`UserAccount.java`, `Role.java`	Учётки и роли VIEWER / OPERATOR / ADMIN
`AuditEvent.java`, `RefreshSession.java`	Аудит и refresh JWT
`AuditedVersionedEntity.java`	created/updated + @Version + Persistable

Базовый класс `AuditedVersionedEntity`: optimistic locking (`@Version`) — при конфликте правок сервер отдаёт **409**, а не перезаписывает чужие изменения. UUID задаётся до `save`, поэтому реализован `Persistable.isNew()` по `version == null`: иначе Spring Data делал бы merge и в ответе уезжала бы устаревшая версия.

Бизнес-правила КР №2 на сервере

1. Граф статусов — `RequestStatus.java`, методы `nextStatuses()` и `canMoveTo()`:

```
NEW → APPROVED | REJECTED
APPROVED → IN_PROGRESS | CANCELLED
IN_PROGRESS → COMPLETED | CANCELLED
COMPLETED, REJECTED, CANCELLED — конечные
```

Повторная установка того же статуса разрешена. Недопустимый переход — `ConflictException` → HTTP 409.

Проверка в `RequestService.changeStatus(...)`. То же правило в bulk и в CSV-предпросмотре (`CsvRequestService`). В ответе API поле `nextStatuses` — UI показывает только допустимые статусы (`RequestMapper` + `static/app.js`).

2. Новая заявка только в NEW — в `RequestService` при create; иначе 400.

3. Дата съёмки не в прошлом — сравнение с `LocalDate.now(clock)`; `Clock` из `ClockConfig.java`.

4. Клиент должен существовать — `ClientService.requireForRequest(clientId)`.

5. Приоритет на сервере — `PriorityCalculator.java` по статусу, бюджету, длительности, флагу urgent (пороги 200k / 500k, 8 / 16 часов). Клиент приоритет не «рисует» сам.

Клиенты как отдельный контур

Раньше ФИО/email/телефон дублировались в заявке. Теперь нормализация:

- API: `ClientController.java` → `/api/clients`
- логика: `ClientService.java` (поиск, CRUD, уникальность email → 409, удаление только без заявок и только ADMIN)
- репозиторий: `ClientRepository.java`
- DTO: `ClientDtos.java`
- UI: вкладка «Клиенты» в `static/index.html` / `app.js`

Чтение — VIEWER+, создание/правка — OPERATOR/ADMIN, удаление — ADMIN. В заявке вместо трёх полей — выбор `clientId` (`RequestDtos.RequestUpsert`).

REST API и контроллеры

Контроллер	Назначение
`AuthController`	login / refresh / logout / me
`RequestController`	CRUD заявок, архив, restore, bulk, CSV
`ClientController`	клиенты
`LocationController`	справочник локаций
`NoteController`	заметки
`AnalyticsController`	сводка
`AdminUserController`, `AuditController`	пользователи и аудит

Ошибки — единый JSON через `GlobalExceptionHandler` и `ApiError`. Корреляция запросов — `CorrelationIdFilter`.

Умный фильтр заявок: `SmartFilterParser` + `RequestSpecifications` + `RequestFilter` — ключи вроде `клиент:`, статус, теги.

CSV: `CsvRequestService` — клиент по email (или создание), бизнес-проверки в preview, корректный повторный импорт экспорта.

Безопасность

`SecurityConfig.java`: API закрыт, публичны login/refresh/logout и health; Swagger — только ADMIN.

JWT: `JwtService`, `JwtAuthenticationFilter` — короткий access, refresh в httpOnly cookie. Политика паролей — `PasswordPolicy`. Ограничение попыток входа — `LoginAttemptService`.

Роли: VIEWER — просмотр; OPERATOR — работа с заявками/клиентами/локациями; ADMIN — пользователи, аудит, жёсткое удаление (если включено флагом).

Web-интерфейс для демонстрации

Статика в `backend/src/main/resources/static/`: не отдельный фронтенд-проект, а UI для проверки API. Вкладки заявок, клиентов, локаций, аналитики, админки; форма заявки с допустимыми статусами и выбором клиента.

Документация: `README_KR3.md`, материалы защиты — `docs/defense-kr3.pdf`.

Тесты

Команда: `cd backend && mvn test`.

Тест	Что доказывает
`RequestStatusTest`	граф переходов
`EntityVersionTest`	версия в ответе = версия в БД
`PriorityCalculatorTest`	приоритет
`SmartFilterParserTest`	разбор фильтра
`PasswordPolicyTest`	пароли
`ContextSmokeTest`	контекст Spring поднимается

Демонстрация (порядок)

1. `docker compose` / `start-compose.sh`, открыть UI.
2. Войти админом, сменить пароль.
3. Вкладка «Клиенты» — seed из V3.
4. «Локации» — типы и коды PAV-1, STUDIO-1, SPB-EMB…
5. «Заявки» — `KR1-001`…; открыть NEW → в списке статусов только APPROVED/REJECTED.
6. Недопустимый переход (например COMPLETED → NEW) → 409.
7. Новая заявка не в NEW / дата в прошлом → 400.
8. Удаление клиента с заявками → 409.
9. Swagger (под ADMIN) — контракт API.
10. По желанию: CSV export → import, аналитика, аудит.

Заключение

В КР №3 backend FrameSpace: схема согласована с КР №1, правила статусов и дат — с КР №2, данные и ограничения централизованы на сервере. Есть миграции с upgrade-путём, JWT и роли, REST, UI для проверки и автотесты на ключевые правила. Backend готов как основа для мобильного клиента в следующей работе.

Спасибо. Готова ответить на вопросы.

5. Карта «где / что / зачем»

Где	Что	Зачем
`domain/RequestStatus.java`	граф статусов	единые переходы на сервере
`service/RequestService.java`	create/update, дата, статус, клиент	бизнес-правила КР №2
`config/ClockConfig.java`	бин Clock	проверяемая «сегодняшняя» дата
`domain/Client.java` + `V2__…sql`	сущность и таблица клиентов	нормализация модели КР №1
`controller/ClientController.java`	/api/clients	CRUD клиентов с ролями
`service/ClientService.java`	уникальный email, запрет удаления с заявками	целостность данных
`V3__kr1_seed_data.sql`	5/8/10 записей KR1	демо сразу после деплоя
`service/PriorityCalculator.java`	расчёт приоритета	одинаковый приоритет для всех клиентов API
`domain/AuditedVersionedEntity.java`	version + Persistable	optimistic lock без ложных 409
`config/SecurityConfig.java` + `security/*`	JWT, роли	защита API
`filter/SmartFilterParser.java`	умный поиск	гибкая фильтрация заявок
`service/CsvRequestService.java`	импорт/экспорт	обмен данными и проверки в preview
`static/index.html`, `app.js`	web UI	демонстрация без отдельного фронта
`error/GlobalExceptionHandler.java`	единый JSON ошибок	предсказуемый контракт
`README_KR3.md`, `docs/defense-kr3.pdf`	описание и защита	документация к работе

6. Типичные вопросы преподавателя

Почему Hibernate validate, а не update?

Схему меняет только Flyway — воспроизводимые миграции и контроль версий БД.

Зачем отдельная таблица client?

В КР №1 клиент — самостоятельная сущность; иначе ФИО/телефон дублируются и нельзя вести справочник заказчиков.

Где граф статусов?

В `RequestStatus`, вызывается из `RequestService` (и CSV/bulk). UI берёт `nextStatuses` из ответа.

Что будет при параллельном редактировании?

`@Version` → 409 «Заявка уже изменена».

Зачем UI в static, если КР про backend?

Для проверки API и демонстрации; отдельный фронтенд по заданию КР №3 не требуется.

Как связаны КР?

V2/V3 и Client — к КР №1; граф и даты — к КР №2; Spring API — КР №3.

Документ для устной защиты КР №3. Сопутствующий презентационный материал: docs/defense-kr3.pdf.